

Ambiente Virtual (venv) e Isolamento de Projetos: Aplicação Prática no VS Code

A organização profissional de projetos em múltiplos arquivos e importações internas estabelece uma base arquitetural onde responsabilidades são separadas, o fluxo é previsível e o código cresce com menor risco de acoplamento indevido — exatamente o tipo de estrutura reforçada no material de modularização e resolução de importações “Estrutura Profissional de Projetos Python: Modularização e Organização no VS Code”. Quando bibliotecas externas entram no projeto, surge um segundo eixo de organização profissional: **o controle e isolamento de dependências**.

Para introduzir instalação de dependência externa com o mínimo de complexidade e sem infraestrutura paralela (banco, web, hardware), a biblioteca escolhida é:

- **NumPy** (primeira biblioteca externa para prática de venv)

Motivo técnico: instala via pip sem exigir configuração extra, funciona imediatamente em um script simples e permite evidenciar isolamento (funciona no venv, pode não funcionar fora dele).

1. Problema Real: versões diferentes no mesmo computador

Um computador pode conter vários projetos Python. Se as bibliotecas forem instaladas “globalmente” (no Python do sistema), aparece um conflito inevitável:

- Projeto A depende de uma versão específica de uma biblioteca.
- Projeto B depende de outra versão da mesma biblioteca.

Como o Python global só pode “ter uma versão ativa por vez” de uma mesma biblioteca em seu diretório principal de pacotes, atualizar ou alterar para atender um projeto pode quebrar outro.

Esse tipo de quebra não costuma ser um erro óbvio. Em muitos casos, o programa roda, mas começa a produzir resultados diferentes, falhas em pontos específicos, ou erros intermitentes por mudança de comportamento interno.

A ideia central é simples: **Ambiente virtual existe para impedir esse conflito**.

2. Conceito de Ambiente Virtual (venv)

Um **ambiente virtual** é uma estrutura local (dentro do projeto) que cria um “Python isolado” para aquele projeto, com:

- um conjunto próprio de bibliotecas instaladas;

- um diretório próprio de pacotes (site-packages);
- um executável/atalho de Python e pip “apontando” para esse conjunto.

Resultado prático:

- o projeto passa a controlar suas bibliotecas sem interferir no Python do sistema;
- a instalação global fica preservada;
- cada projeto pode ter suas versões, seus pacotes e seu histórico de dependências.

A ideia central é simples: **bibliotecas externas deixam de ser “do computador” e passam a ser “do projeto”**.

3. Estrutura de projeto com venv

Estrutura típica:

```
meu_projeto/  
|  
├── venv/  
├── main.py  
└── requirements.txt
```

3.1. O que é a pasta venv/

venv/ é a “caixa isolada” do projeto. Ela contém arquivos internos que permitem executar Python e instalar bibliotecas sem tocar a instalação global.

Dentro dela existem subpastas e arquivos que variam entre Windows e Linux/macOS, mas o papel é o mesmo: manter o ambiente isolado.

3.2. O que é site-packages

site-packages é a pasta onde as bibliotecas externas ficam instaladas.

- Sem ambiente virtual: site-packages pertence ao Python global.
- Com ambiente virtual: site-packages pertence ao projeto (dentro de venv/).

Isso é o coração do isolamento. Quando `pip install ...` é executado com o ambiente ativado, a biblioteca é instalada no **site-packages do venv**, não no do sistema.

3.3. Ligação direta com sys.path

sys.path é a lista de diretórios onde o interpretador procura módulos e pacotes durante um import.

Quando o ambiente virtual está ativo, o Python passa a priorizar caminhos do próprio ambiente, incluindo o site-packages do venv. Isso faz com que:

- import numpy funcione dentro do ambiente, pois o pacote foi instalado ali;
- fora do ambiente, pode não funcionar (se não existir numpy instalado globalmente).

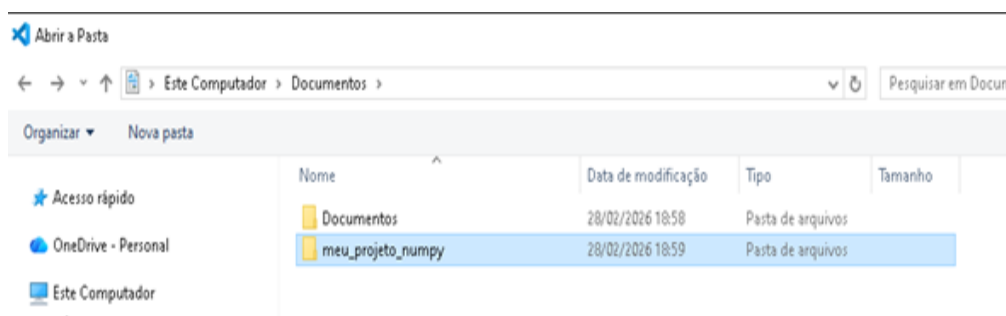
Nenhuma “nova regra” aparece: **o mecanismo do import é o mesmo**. O que muda é o **caminho preferencial** onde o Python procura.

4. Preparação do projeto no VS Code

4.1. Criar pasta do projeto

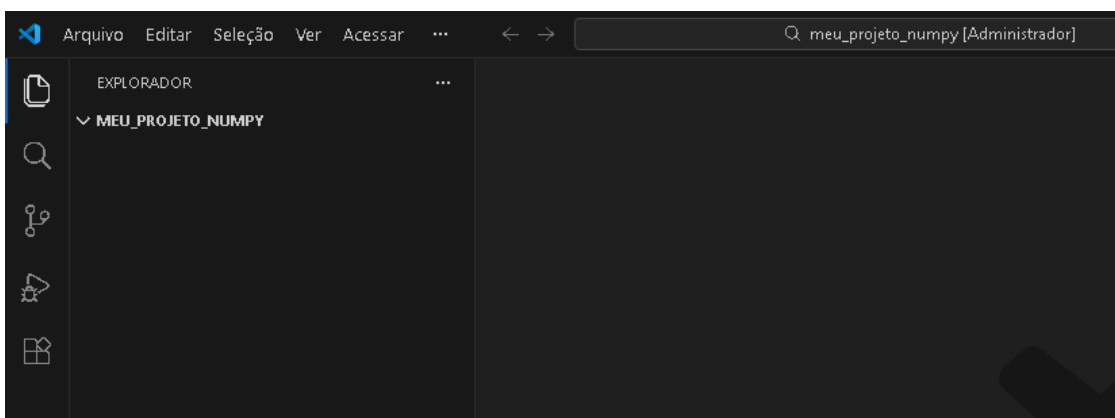
Criar uma pasta para o projeto, por exemplo:

- meu_projeto_numpy



Abrir essa pasta no VS Code:

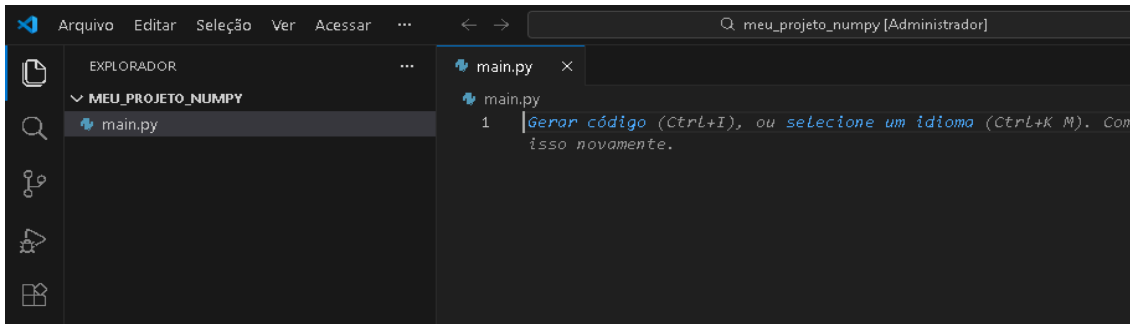
- VS Code → File → Open Folder... → selecionar meu_projeto_numpy



A abertura por pasta é importante porque o terminal do VS Code costuma iniciar “no diretório correto”, evitando comandos executados no lugar errado.

4.2. Criar o arquivo principal

Criar main.py dentro da pasta do projeto.

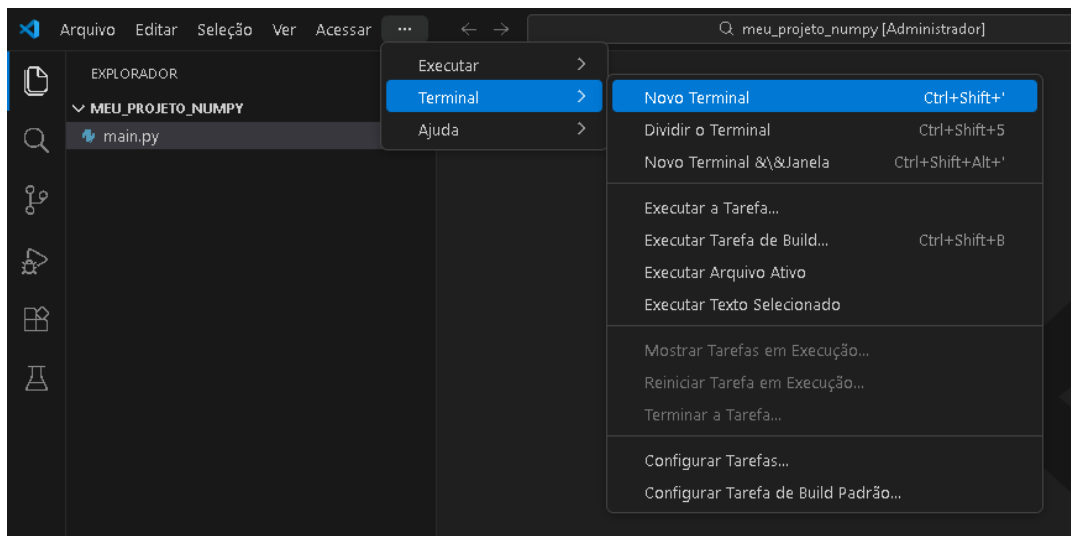


Por enquanto, pode ficar vazio. O foco inicial é preparar ambiente.

5. Criar ambiente virtual (venv)

Abrir o terminal integrado do VS Code:

- Menu → Terminal → New Terminal



Garantir que o terminal está no diretório do projeto. Em geral, o terminal mostra o caminho atual. Se estiver fora, o comando será executado em outro local e o venv/ aparecerá na pasta errada.

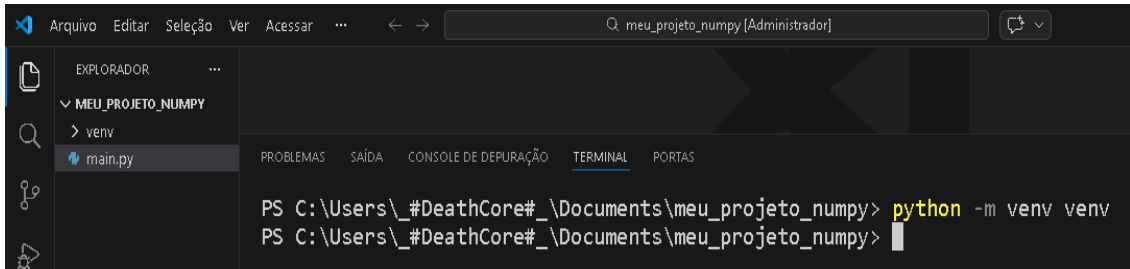
5.1. Comando de criação

Executar:

```
python -m venv venv
```



A screenshot of the Visual Studio Code interface. The Explorer pane on the left shows a project named 'MEU_PROJETO_NUMPY' with a file 'main.py'. The Terminal pane at the bottom shows a PowerShell prompt 'PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy>' with the command 'python -m venv venv' being entered. The command is highlighted in yellow.



A screenshot of the Visual Studio Code interface. The Explorer pane on the left shows a project named 'MEU_PROJETO_NUMPY' with a file 'main.py'. The Terminal pane at the bottom shows a PowerShell prompt 'PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy>' with the command 'python -m venv venv' being entered. The command is highlighted in yellow.

Explicação detalhada do comando:

- python chama o interpretador Python instalado no sistema (neste momento, o global).
- -m venv manda o Python executar um **módulo interno** chamado venv, responsável por criar ambientes virtuais.
- venv (no final) é o nome da pasta que será criada contendo o ambiente.

Ao final, uma pasta venv/ aparece dentro do projeto.

5.2. O que acabou de acontecer

O Python criou uma estrutura interna com:

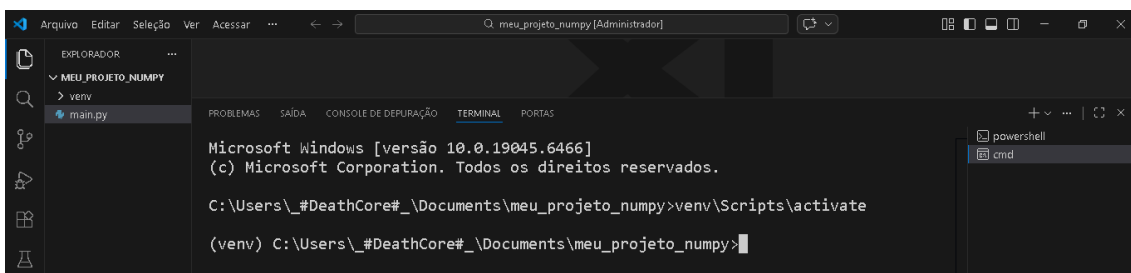
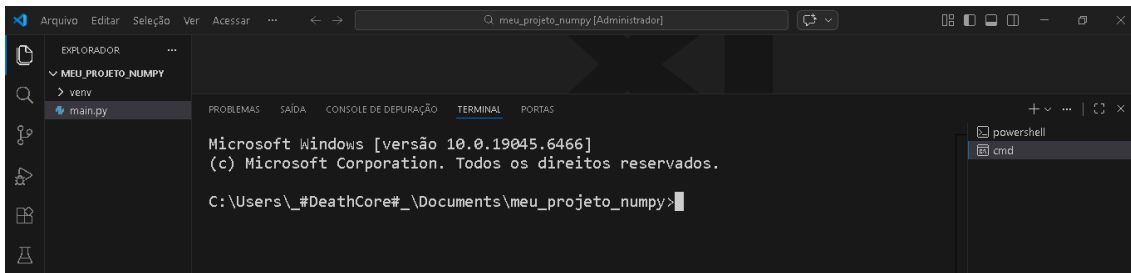
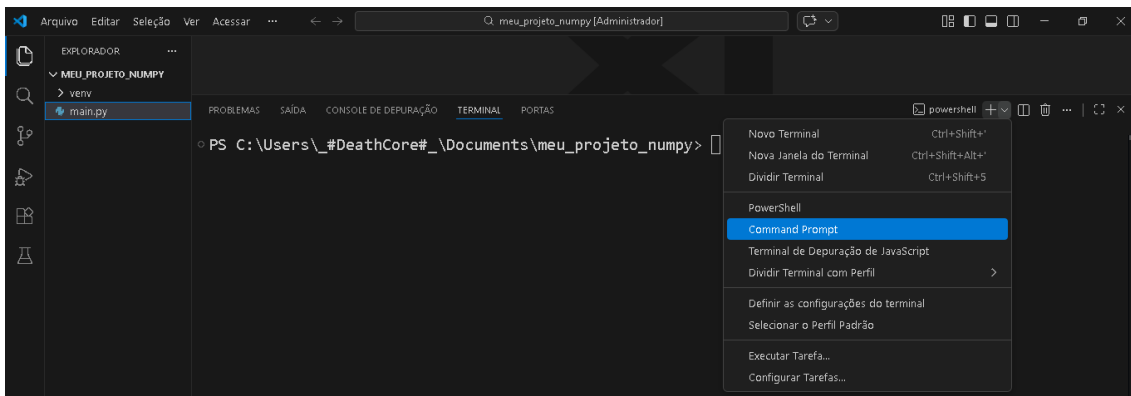
- ferramentas para ativar o ambiente;
- uma referência para um interpretador isolado;
- um local próprio para bibliotecas.

Ainda não há nenhuma biblioteca externa instalada. O ambiente foi apenas criado.

6. Ativar o ambiente virtual (Windows / VS Code)

6.1. Comando de ativação (Windows)

No terminal, executar: venv\Scripts\activate



6.2. Como saber se ativou corretamente

Normalmente aparece uma indicação no início da linha do terminal, algo como:

- (venv)

Isso significa que o terminal “entrou” no contexto do ambiente virtual. A partir desse ponto:

- python passa a apontar para o Python do ambiente.
- pip passa a apontar para o pip do ambiente.

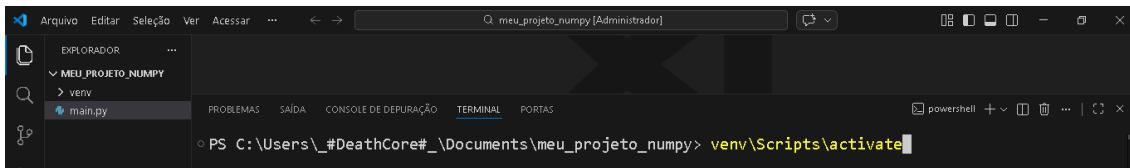
Essa troca é o que garante que bibliotecas instaladas fiquem isoladas.

6.3. Problema comum: execução bloqueada no PowerShell

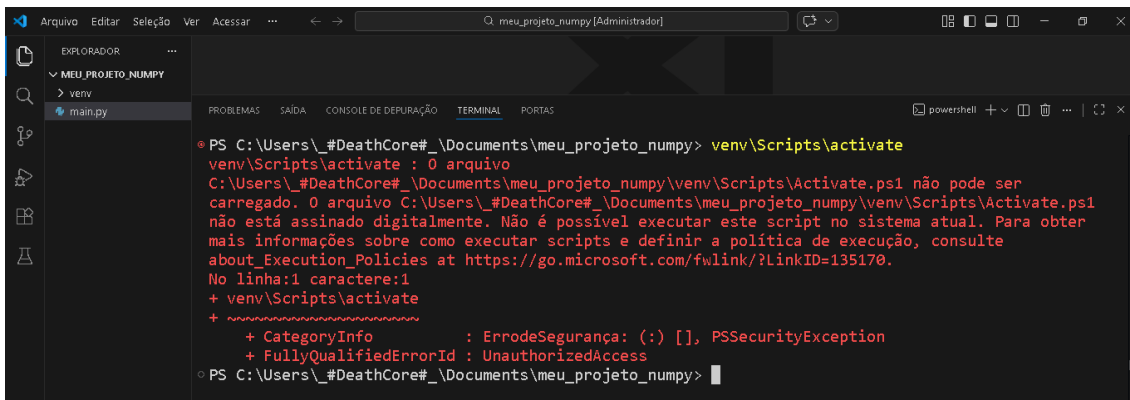
Em alguns computadores, o PowerShell bloqueia scripts. Se surgir erro relacionado a execução de scripts, existe duas saídas simples (sem entrar em conteúdo avançado):

1. trocar o terminal do VS Code para **Command Prompt (cmd)**
2. ativar via cmd

Como o objetivo aqui é manter a aula fluida, a recomendação prática é: **usar cmd se houver bloqueio**. O comando de ativação continua o mesmo.



```
PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy> venv\Scripts\activate
```



```
PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy> venv\Scripts\activate
venv\Scripts\activate : 0 arquivo
C:\Users\#DeathCore#\Documents\meu_projeto_numpy\venv\Scripts\Activate.ps1 não pode ser
carregado. O arquivo C:\Users\#DeathCore#\Documents\meu_projeto_numpy\venv\Scripts\Activate.ps1
não está assinado digitalmente. Não é possível executar este script no sistema atual. Para obter
mais informações sobre como executar scripts e definir a política de execução, consulte
about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
No linha:1 caractere:1
+ venv\Scripts\activate
+ ~~~~~
+ CategoryInfo          : ErroDeSegurança: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy>
```

7. Instalação de biblioteca externa (NumPy) — explicação completa, sem economia

7.1. O que significa “instalar uma biblioteca” de verdade

Instalar uma biblioteca não é “baixar um arquivo solto”. Instalar uma biblioteca significa:

1. **obter o pacote** (normalmente do repositório Python Package Index — PyPI);
2. **colocar esse pacote** dentro do diretório correto de bibliotecas do Python ativo;
3. registrar metadados (nome, versão, dependências) para o interpretador reconhecer o pacote.

O pip é o instalador que automatiza isso.

Quando o ambiente virtual está ativo, o “Python ativo” é o do venv. Então:

- o pip instala no site-packages do venv;
- não instala no Python global.

Essa distinção é o ponto mais importante da aula prática.

7.2. O que é pip (definição objetiva)

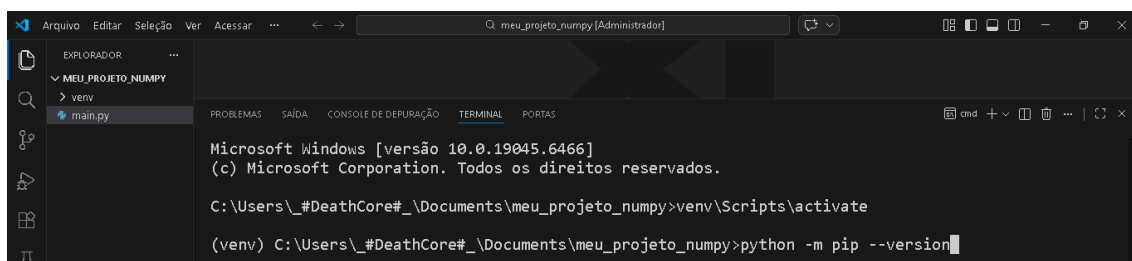
pip é o gerenciador de pacotes do ecossistema Python.

Funções principais:

- instalar bibliotecas externas;
- remover bibliotecas;
- listar bibliotecas instaladas;
- exportar dependências para arquivo (requirements.txt).

7.3. Confirmar que pip está apontando para o venv (checagem recomendada)

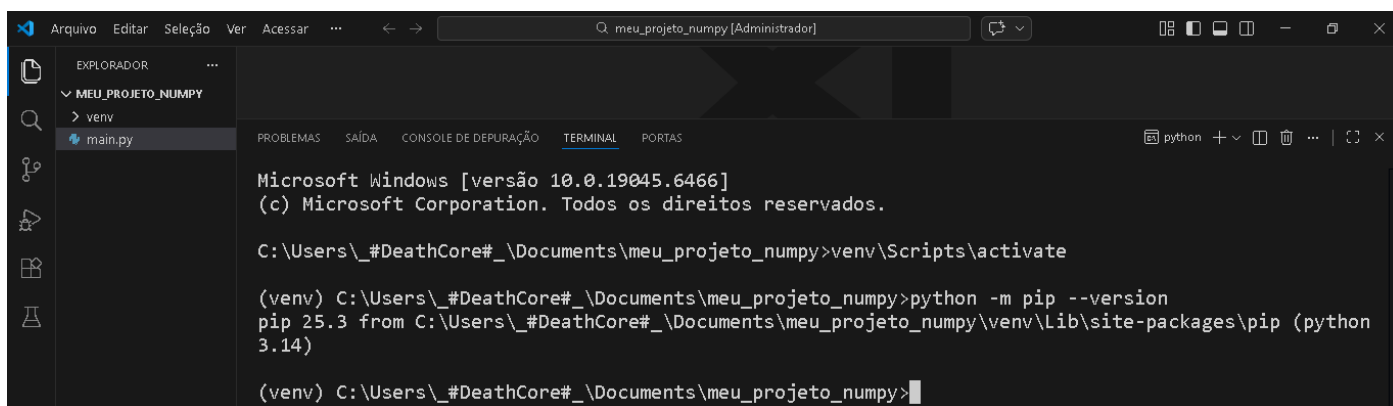
Com o ambiente ativado, executar: `python -m pip --version`



```
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\#DeathCore#\Documents\meu_projeto_numpy>venv\Scripts\activate

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>python -m pip --version
```



```
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\#DeathCore#\Documents\meu_projeto_numpy>venv\Scripts\activate

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>python -m pip --version
pip 25.3 from C:\Users\#DeathCore#\Documents\meu_projeto_numpy\venv\Lib\site-packages\pip (python
3.14)

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>
```

Como interpretar o resultado:

O terminal exibirá algo como:

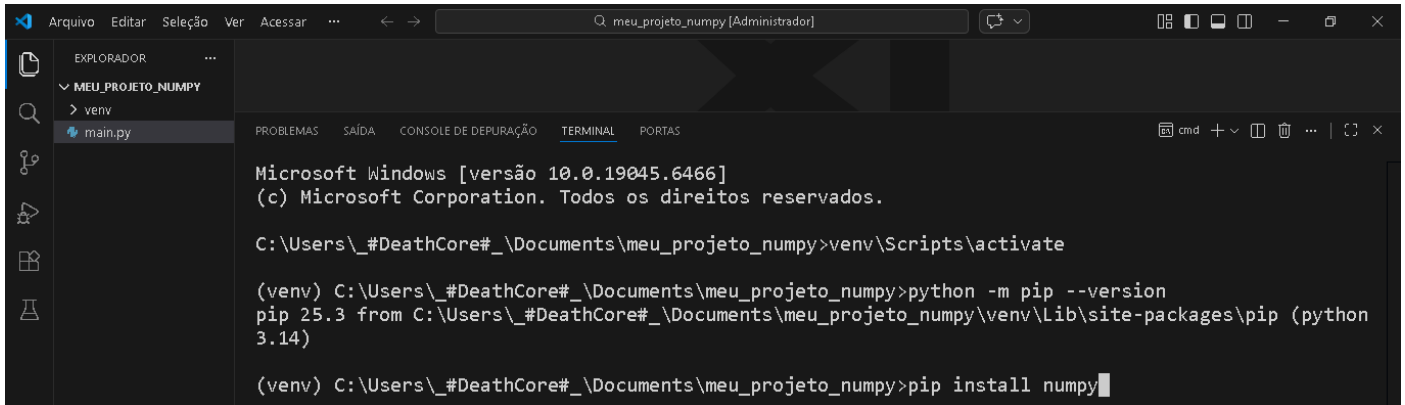
- versão do pip
- caminho do pip

Se o caminho exibido mencionar a pasta do projeto e venv, então o pip está ligado ao ambiente isolado.

Essa checagem é simples e evita um erro clássico: instalar pacotes fora do venv sem perceber.

7.4. Instalar NumPy

Com o ambiente ativo (venv), executar: `pip install numpy`

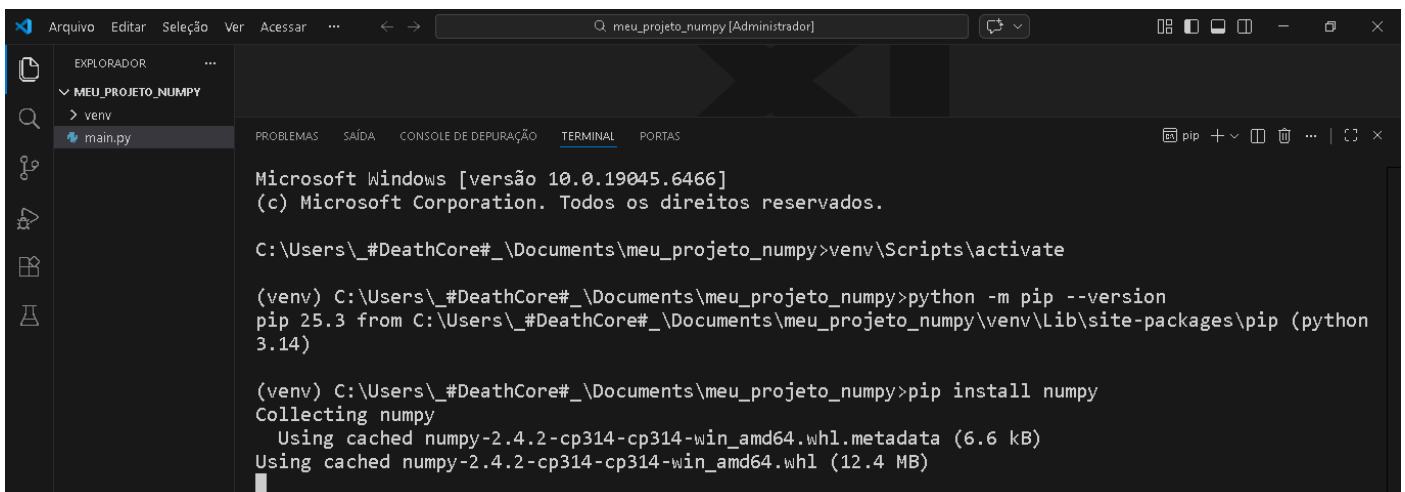


```
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\#DeathCore#\Documents\meu_projeto_numpy>venv\Scripts\activate

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>python -m pip --version
pip 25.3 from C:\Users\#DeathCore#\Documents\meu_projeto_numpy\venv\Lib\site-packages\pip (python 3.14)

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>pip install numpy
```



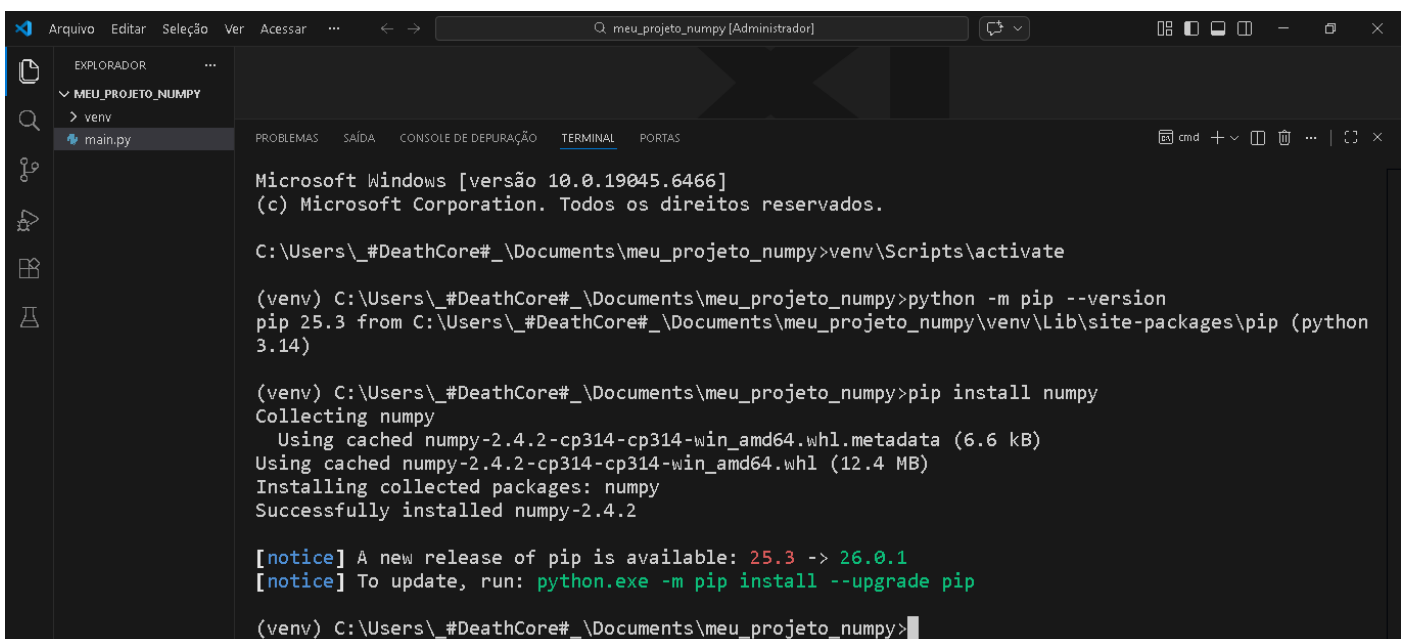
```
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\#DeathCore#\Documents\meu_projeto_numpy>venv\Scripts\activate

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>python -m pip --version
pip 25.3 from C:\Users\#DeathCore#\Documents\meu_projeto_numpy\venv\Lib\site-packages\pip (python 3.14)

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>pip install numpy
Collecting numpy
  Using cached numpy-2.4.2-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
  Using cached numpy-2.4.2-cp314-cp314-win_amd64.whl (12.4 MB)

```



```
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\#DeathCore#\Documents\meu_projeto_numpy>venv\Scripts\activate

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>python -m pip --version
pip 25.3 from C:\Users\#DeathCore#\Documents\meu_projeto_numpy\venv\Lib\site-packages\pip (python 3.14)

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>pip install numpy
Collecting numpy
  Using cached numpy-2.4.2-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
  Using cached numpy-2.4.2-cp314-cp314-win_amd64.whl (12.4 MB)
Installing collected packages: numpy
Successfully installed numpy-2.4.2

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>
```

➤ O que acontece nos bastidores

Ao executar `pip install numpy`, o processo típico é:

1. Resolução do pacote

- o pip identifica o pacote chamado `numpy` no repositório.
- verifica versões disponíveis e escolhe a mais adequada para o ambiente (Python e sistema operacional).

2. Download

- o pip baixa o pacote, geralmente em formato já preparado para instalação.

3. Instalação no local correto

- o pip copia/instala o pacote no `site-packages` do **ambiente virtual**.

4. Registro

- o pip registra a versão instalada e os metadados do pacote.
- isso faz com que `import numpy` seja reconhecido pelo interpretador do ambiente.

➤ Por que isso prova o isolamento?

Porque toda essa instalação ocorre *dentro da pasta do projeto*, na estrutura do `venv/`.

Se outro projeto tiver outro `venv`, a instalação será independente.

Se o ambiente for desativado, o Python global pode não enxergar `NumPy`.

7.5. Verificar se NumPy realmente foi instalado no ambiente

Existem algumas verificações simples (sem aprofundar em ferramentas além do necessário):

➤ Opção A — listar pacotes instalados

`pip list`

Procurar `numpy` na lista.

```
(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>pip list
Package Version
-----
numpy    2.4.2
pip      25.3

(venv) C:\Users\#DeathCore#\Documents\meu_projeto_numpy>
```

➤ Opção B — mostrar informações do pacote

pip show numpy

Esse comando mostra:

- nome do pacote
- versão
- local onde foi instalado

```
(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>pip show numpy
Name: numpy
Version: 2.4.2
Summary: Fundamental package for array computing in Python
Home-page: https://numpy.org
Author: Travis E. Oliphant et al.
Author-email:
License-Expression: BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0
Location: C:\Users\_#DeathCore#\Documents\meu_projeto_numpy\venv\Lib\site-packages
Requires:
Required-by:

(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>
```

O campo **Location** costuma apontar diretamente para o site-packages do ambiente virtual. Isso é uma evidência concreta de isolamento.

8. Teste mínimo de uso no main.py

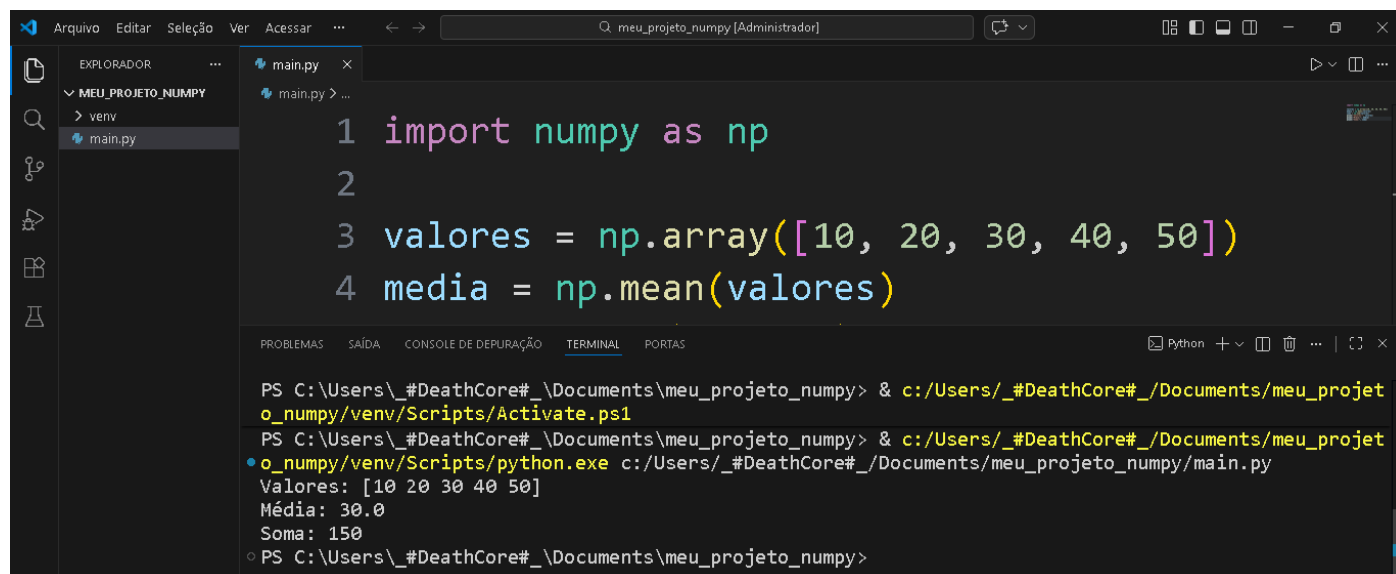
Abrir main.py e inserir um script curto e direto, apenas para provar que:

- import funciona
- o pacote está operacional

A screenshot of a code editor window titled 'meu_projeto_numpy [Administrador]'. The left sidebar shows a file explorer with the following structure: 'MEU_PROJETO_NUMPY' containing 'venv' and 'main.py'. The 'main.py' file is selected and open in the editor. The code in the editor is as follows:

```
1 import numpy as np
2
3 valores = np.array([10, 20, 30, 40, 50])
4 media = np.mean(valores)
5 soma = np.sum(valores)
6 print("Valores:", valores)
7 print("Média:", media)
8 print("Soma:", soma)
9
```

Se o programa rodar e imprimir valores, NumPy está instalado e acessível dentro do ambiente.



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'MEU_PROJETO_NUMPY' with a subdirectory 'venv' containing 'main.py'. The main editor displays the following Python code in 'main.py':

```
1 import numpy as np
2
3 valores = np.array([10, 20, 30, 40, 50])
4 media = np.mean(valores)
```

The TERMINAL pane at the bottom shows the execution of the script. The commands entered are:

```
PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy> & c:/Users/_#DeathCore#/_Documents/meu_projeto_numpy/venv/Scripts/Activate.ps1
PS C:\Users\#DeathCore#\Documents\meu_projeto_numpy> & c:/Users/_#DeathCore#/_Documents/meu_projeto_numpy/venv/Scripts/python.exe c:/Users/_#DeathCore#/_Documents/meu_projeto_numpy/main.py
```

The output of the script is:

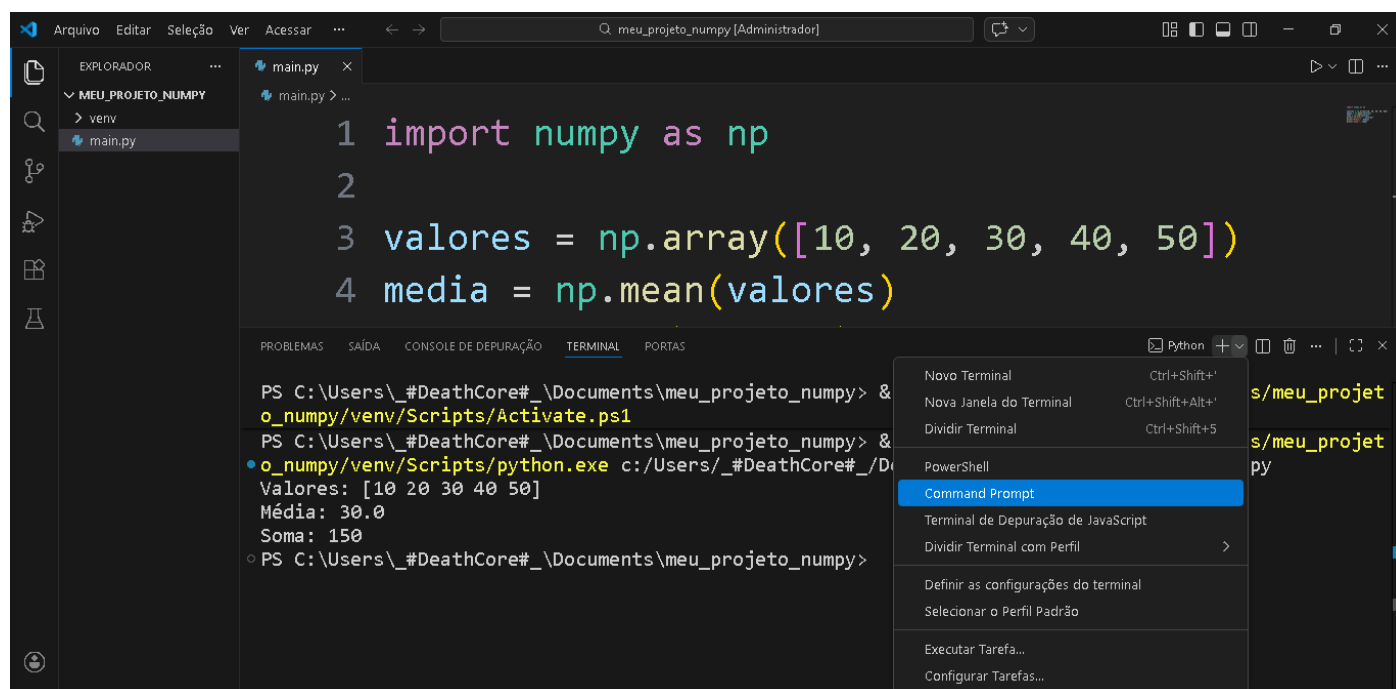
```
Valores: [10 20 30 40 50]
Média: 30.0
Soma: 150
```

9. Demonstrar o isolamento (prova prática do conceito)

9.1. Desativar o ambiente

deactivate

A indicação (venv) some do terminal.



This screenshot shows the same VS Code interface as before, but with a context menu open over the terminal. The terminal shows the same code and output as the previous screenshot. The context menu, which appears after right-clicking in the terminal, includes the following options:

- Novo Terminal (Ctrl+Shift+')
- Nova Janela do Terminal (Ctrl+Shift+Alt+')
- Dividir Terminal (Ctrl+Shift+5)
- PowerShell
- Command Prompt** (highlighted)
- Terminal de Depuração de JavaScript
- Dividir Terminal com Perfil
- Definir as configurações do terminal
- Selecionar o Perfil Padrão
- Executar Tarefa...
- Configurar Tarefas...

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPUÇÃO  TERMINAL  PORTAS

Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>c:/Users/_#DeathCore#/Documents\meu_projeto_numpy/venv/Scripts/activate.bat

(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>deactivate
```

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPUÇÃO  TERMINAL  PORTAS

Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>c:/Users/_#DeathCore#/Documents\meu_projeto_numpy/venv/Scripts/activate.bat

(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>deactivate
C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>
```

9.2. Executar novamente

python main.py

Se NumPy **não estiver instalado globalmente**, é comum aparecer:

- `ModuleNotFoundError: No module named 'numpy'`

```
Arquivo  Editar  Seleção  Ver  Acessar  ...  Python Teste [Administrador]

main.py 1 x
main.py > ...

1 import numpy as np
2
3 valores = np.array([10, 20, 30, 40, 50])
4 media = np.mean(valores)
5 soma = np.sum(valores)

PROBLEMAS 1  SAÍDA  CONSOLE DE DEPUÇÃO  TERMINAL  PORTAS

PS C:\Users\_#DeathCore#\Documents\Documentos\Python Teste> & C:/Users/_#DeathCore#/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/_#DeathCore#/Documents/Documentos/Python Teste/main.py"
Traceback (most recent call last):
  File "c:/Users/_#DeathCore#\Documents\Documentos\Python Teste\main.py", line 1, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
```

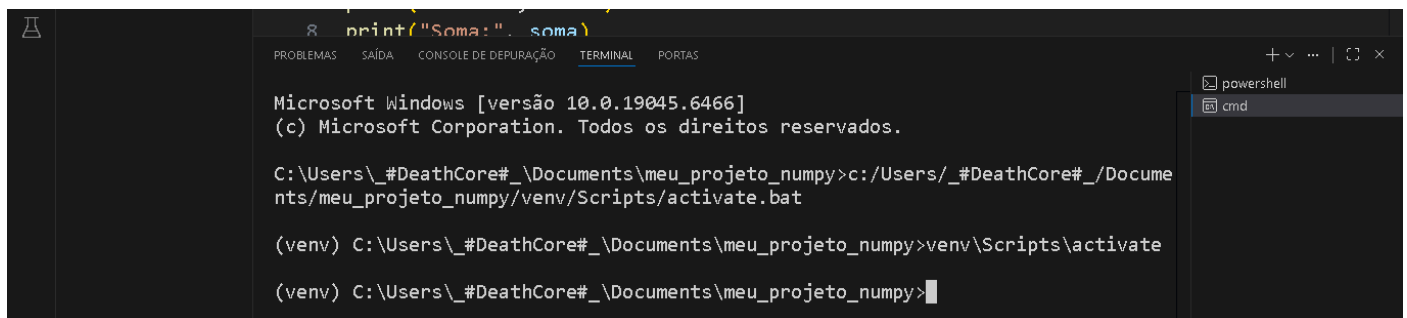
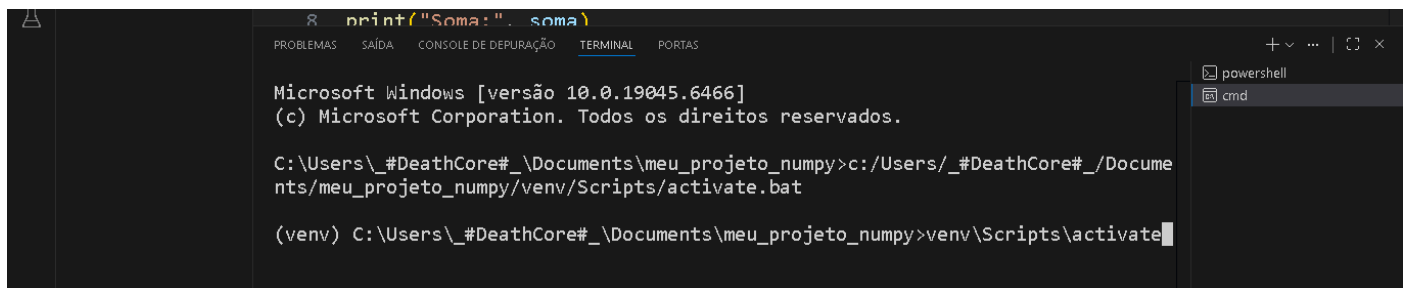
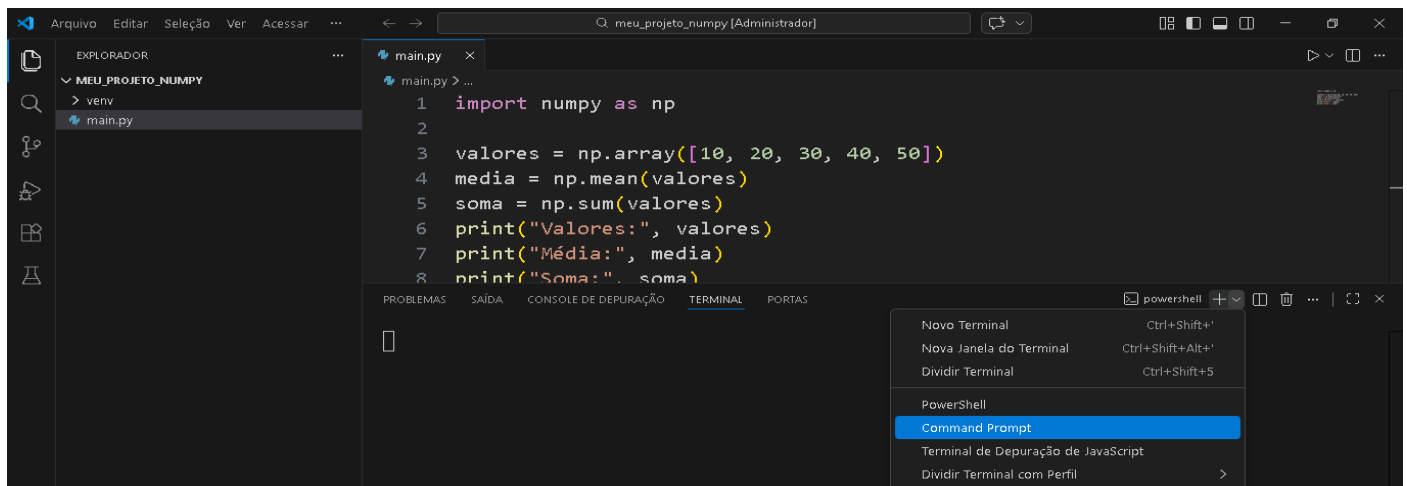
➤ Interpretação:

- NumPy não “sumiu”.
- NumPy continua instalado dentro do venv.
- Apenas o Python ativo agora é o global, que não está apontando para o site-packages do venv.

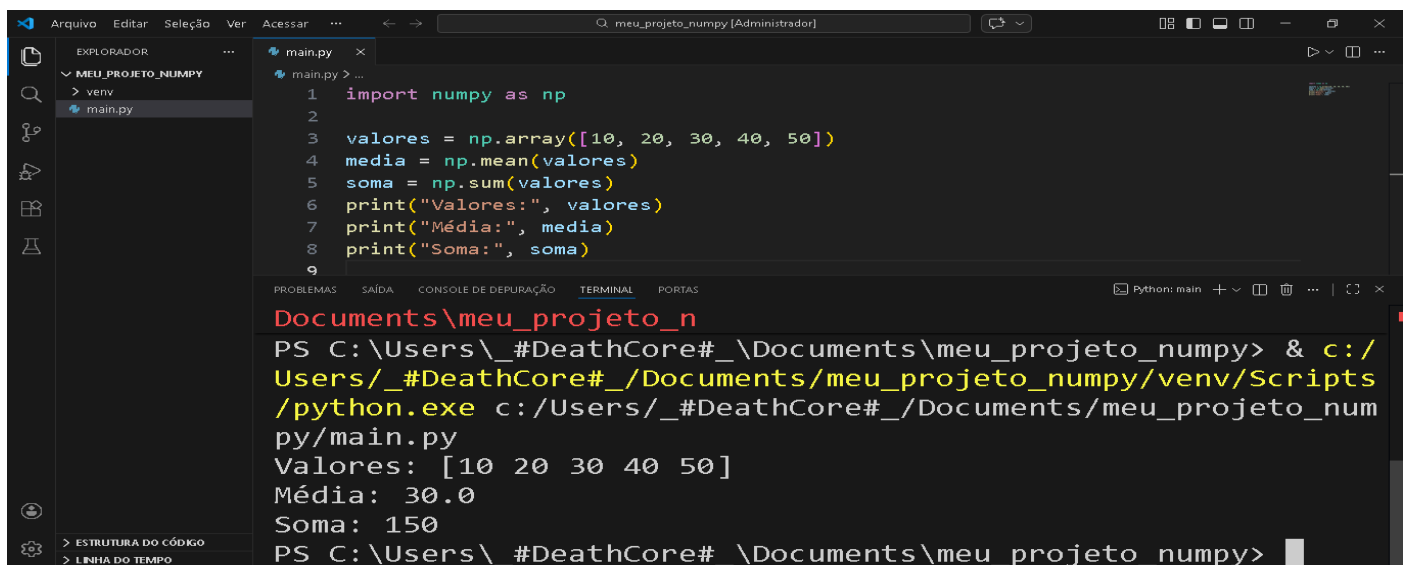
Isso materializa o conceito de isolamento com clareza.

9.3. Reativar e executar de novo

venv\Scripts\activate



python main.py



O código volta a funcionar.

Essa alternância (fora do venv dá erro, dentro do venv funciona) é a demonstração mais didática de isolamento sem entrar em conteúdos paralelos.

10. requirements.txt — registro de dependências

10.1. Por que requirements.txt existe

Um projeto profissional não depende apenas do código.

Ele depende também das bibliotecas e versões que o código pressupõe.

Se outro computador (ou outra pessoa) precisar rodar o mesmo projeto, é necessário:

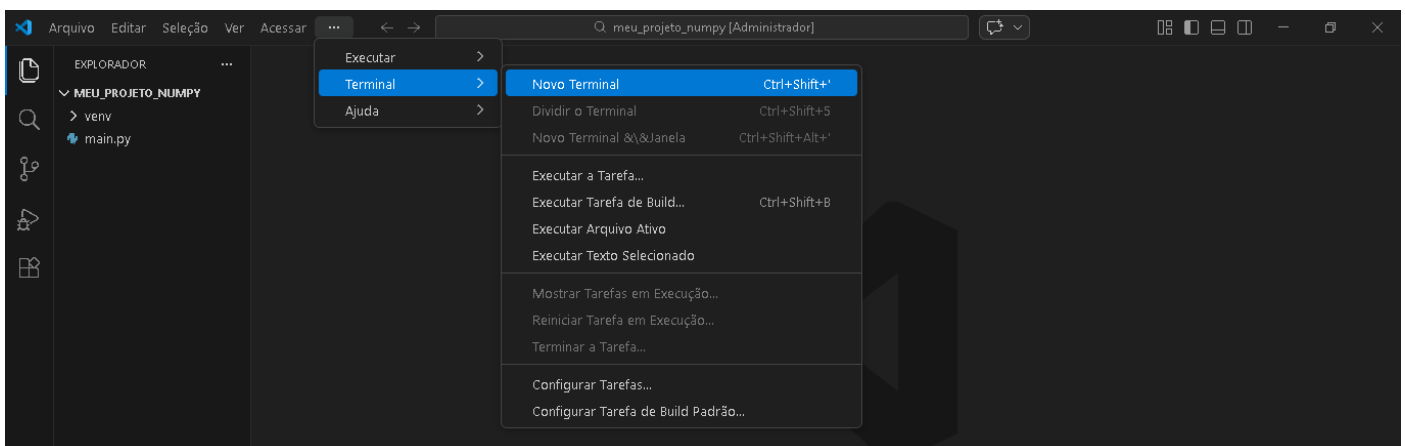
- instalar exatamente as mesmas dependências
- com versões compatíveis

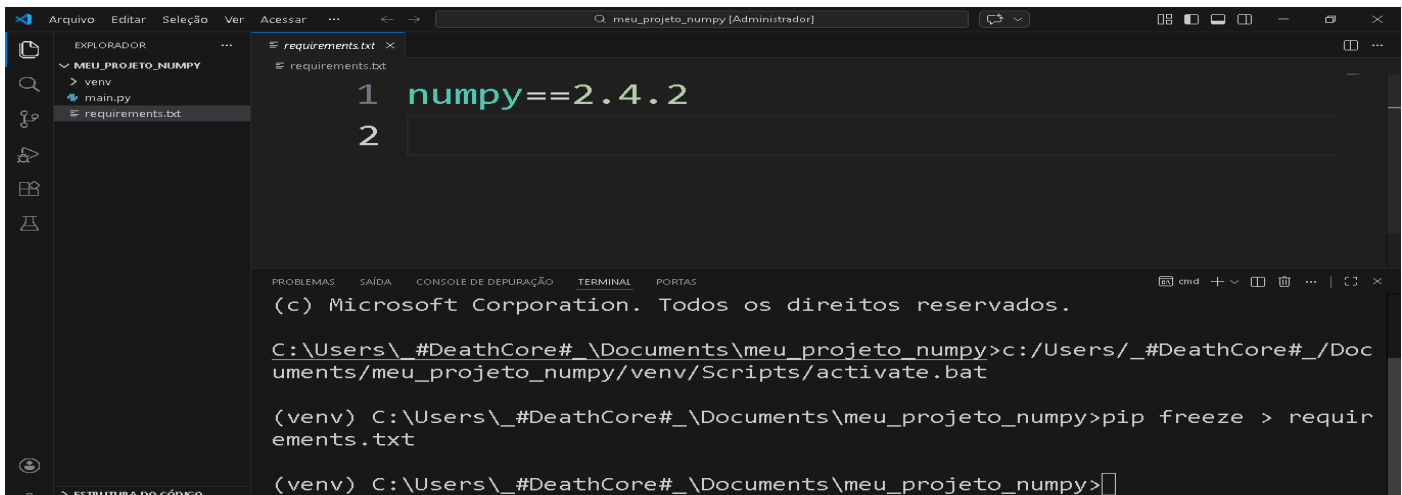
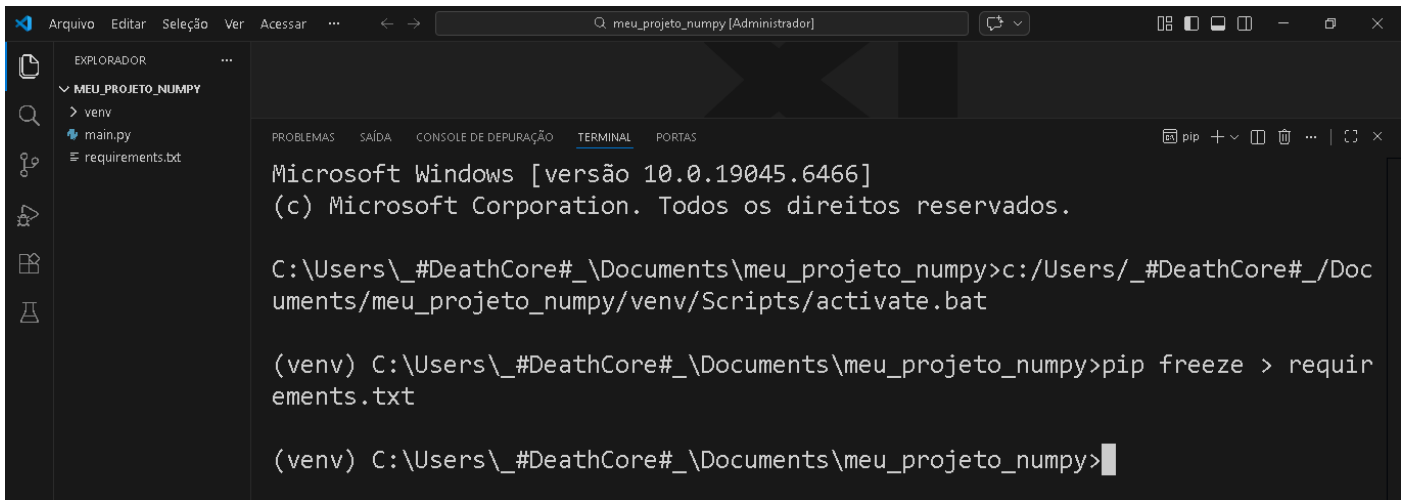
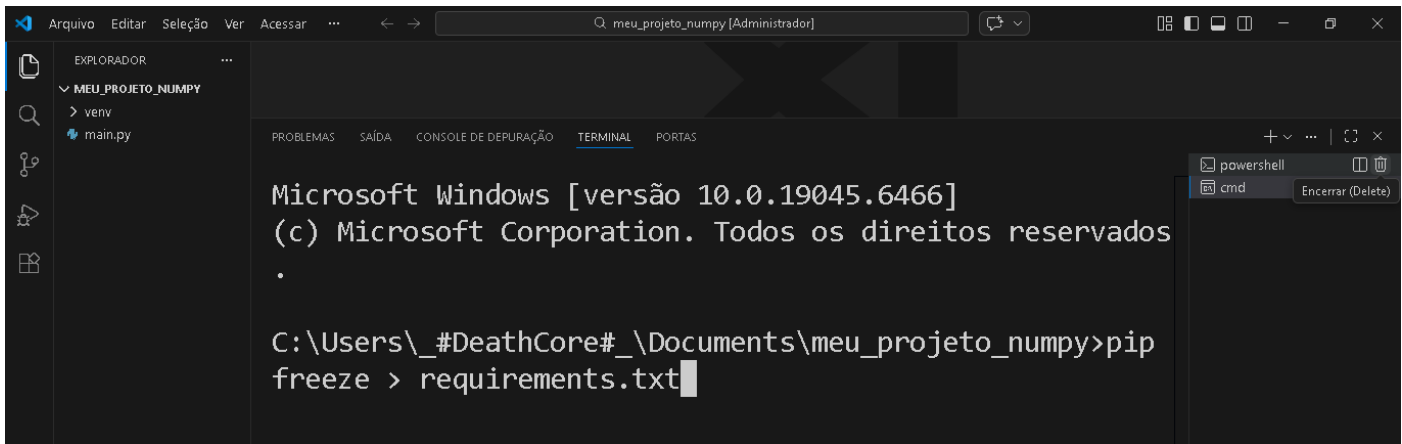
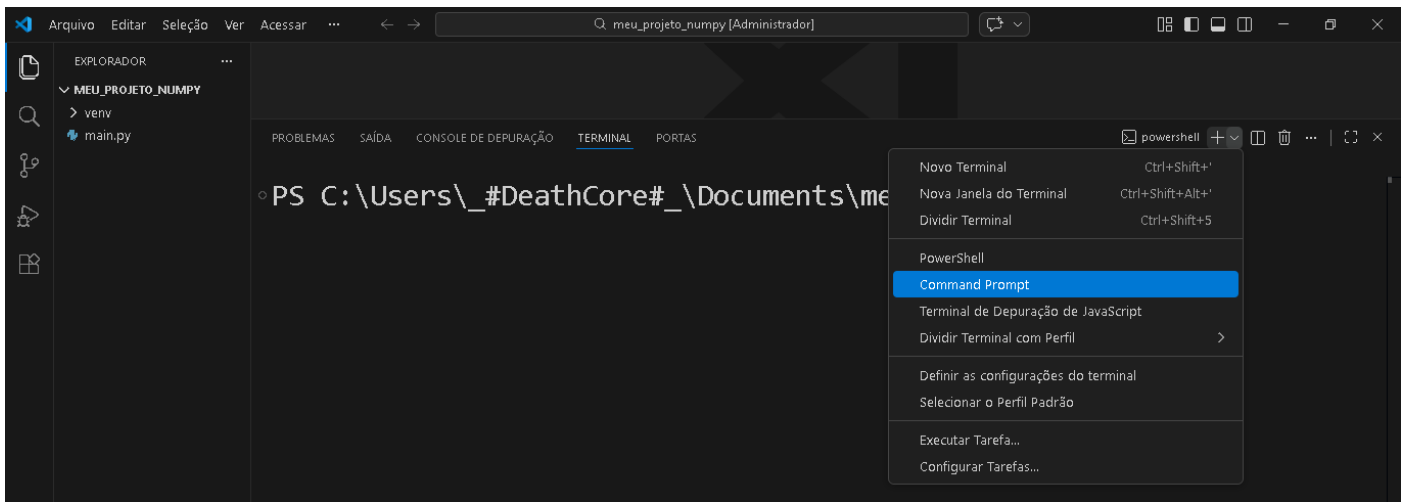
requirements.txt é o arquivo que descreve “o que precisa estar instalado”.

10.2. Gerar requirements.txt (Gerar automaticamente)

Observação: Ativar o venv, se ainda não estiver ativo: `venv\Scripts\activate`

Gerar o arquivo: `pip freeze > requirements.txt`





Explicação detalhada do comando

- **pip freeze** lista pacotes instalados no ambiente **em formato padronizado**, como:
 - `numpy==<versão>`
- `>` redireciona a saída do terminal para um arquivo.
- **requirements.txt** recebe essa lista.

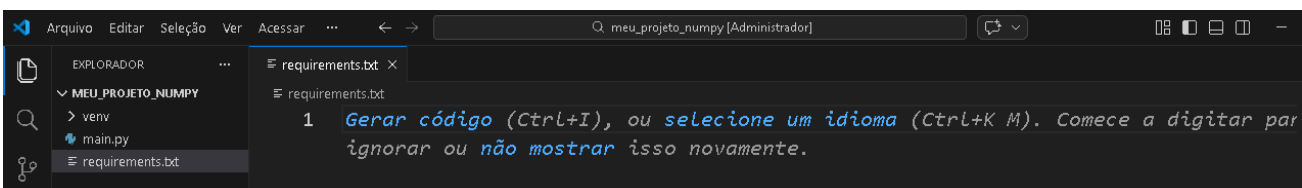
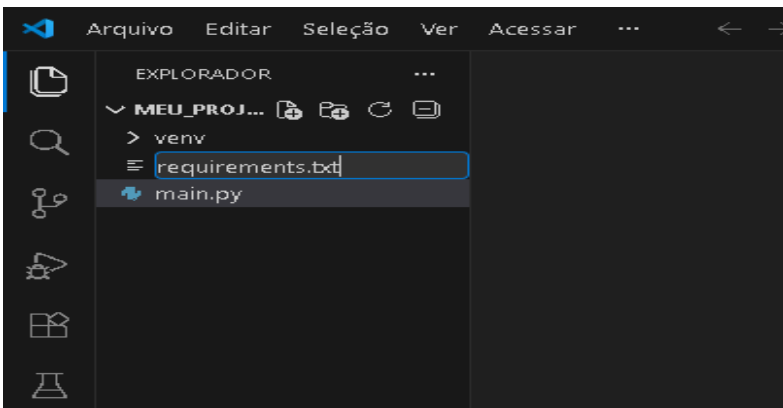
Esse arquivo se torna a “lista oficial de dependências” do projeto.

10.3. Instalar a partir de requirements.txt (manualmente)

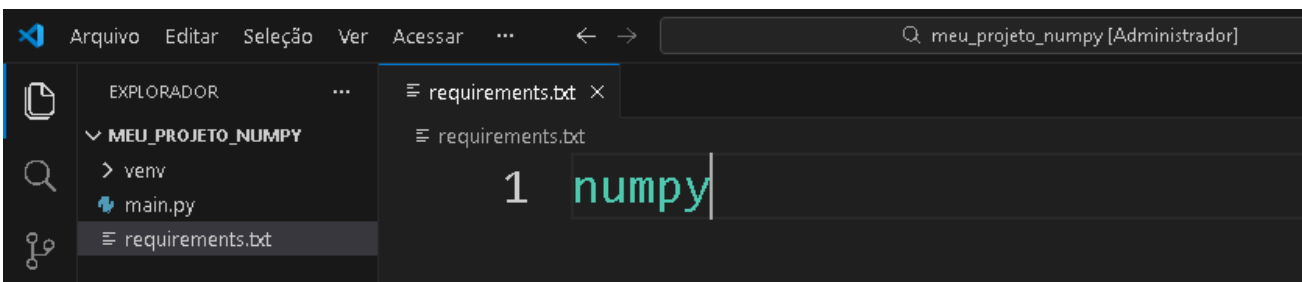
- Criar o arquivo requirements.txt manualmente

Novo arquivo

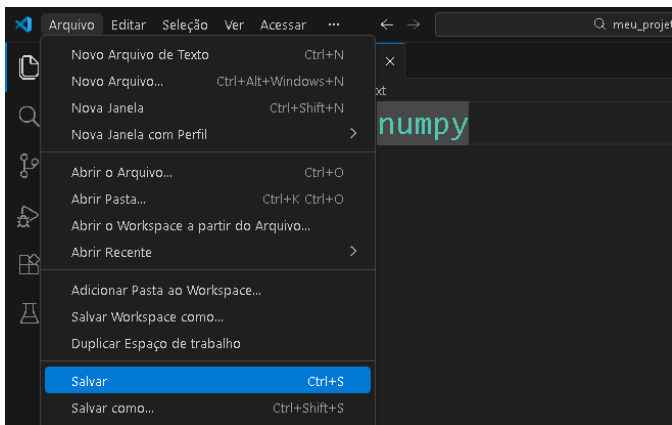
Nomeie como: requirements.txt



Dentro dele escreva: numpy

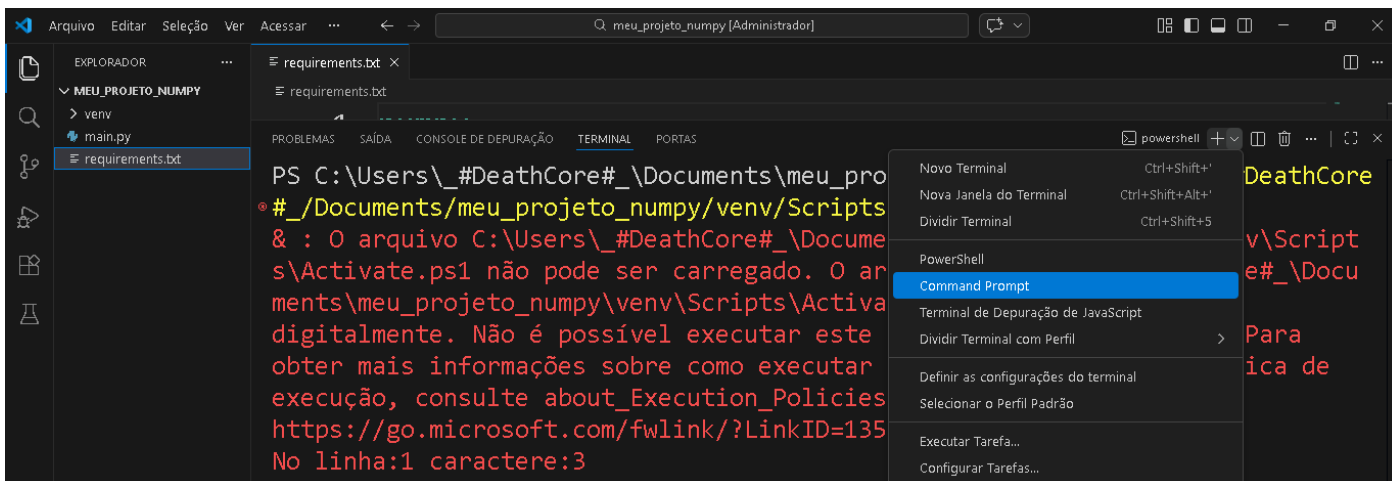
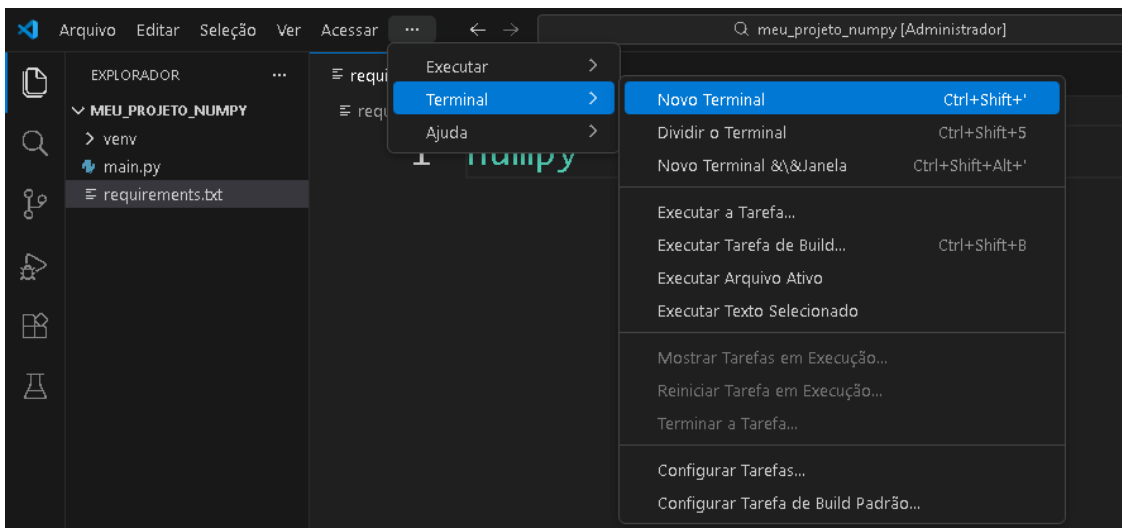


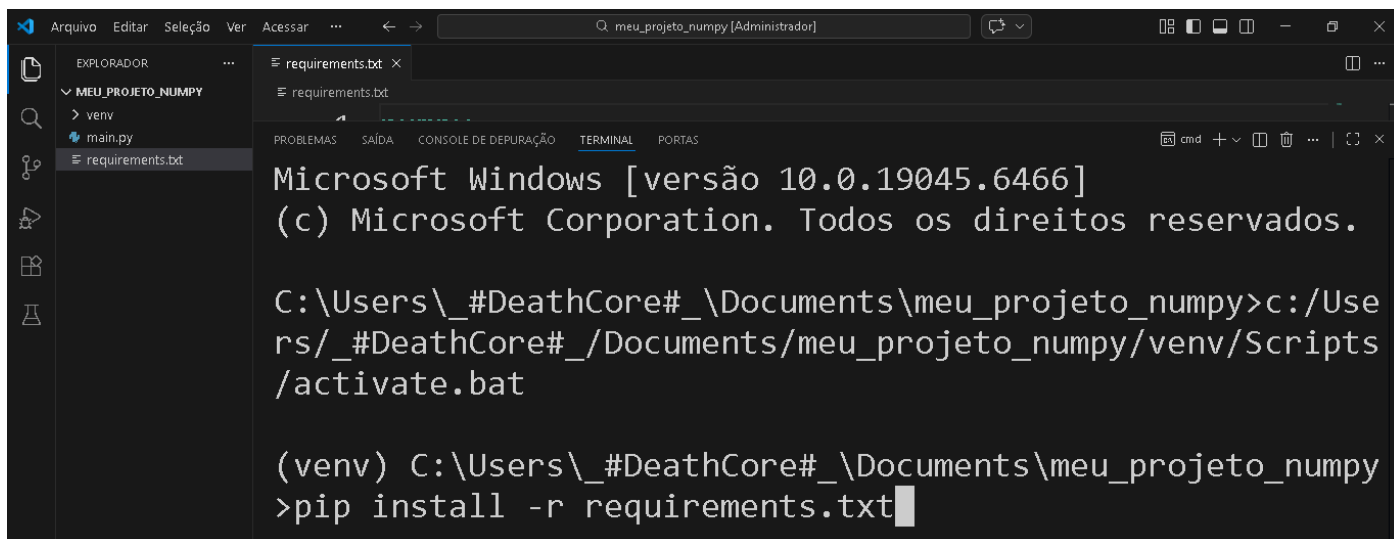
Salve



Em outro ambiente limpo, seria possível reconstruir as dependências com:

`pip install -r requirements.txt`

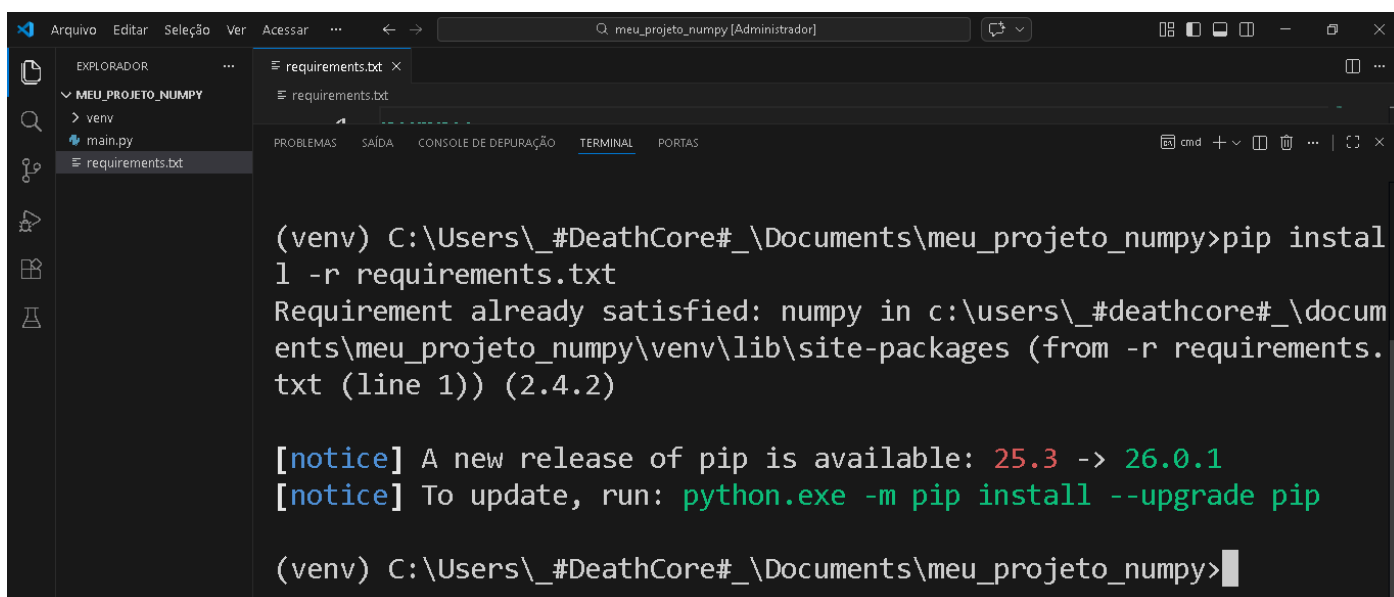




```
Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>c:/Users/_#DeathCore#/Documents/meu_projeto_numpy/venv/Scripts/activate.bat

(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>pip install -r requirements.txt
```



```
(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>pip install -r requirements.txt
Requirement already satisfied: numpy in c:\users\_#deathcore#\documents\meu_projeto_numpy\venv\lib\site-packages (from -r requirements.txt (line 1)) (2.4.2)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\_#DeathCore#\Documents\meu_projeto_numpy>
```

Isso garante reprodutibilidade.

Sem isso, cada máquina vira um “caso diferente”, e o projeto começa a falhar por diferenças de ambiente.

10.4. requirements.txt: Manual vs Automático (pip freeze) — Qual é “melhor” e por quê?

Em Python, o arquivo **requirements.txt** existe para registrar as **dependências** (bibliotecas) necessárias para um projeto funcionar em outro computador, em outro laboratório, ou na máquina de um colega. A questão não é “qual método é certo” de forma absoluta, e sim **qual método é mais adequado para o objetivo**: ensino, reprodutibilidade, padronização ou manutenção profissional.

10.4.1. Criar o requirements.txt manualmente (ex.: numpy)

Como fica:

- Você escreve só:
 - numpy

O que isso significa tecnicamente:

- Você está dizendo: “este projeto precisa do NumPy”, mas **não está amarrando** uma versão específica.
- Quando alguém executar `pip install -r requirements.txt`, o pip vai instalar **a versão mais recente compatível** naquele momento (ou alguma versão que ele resolva como melhor, dependendo do ambiente).

Vantagens (por que isso é bom):

- **Didático e limpo:** para aula, é perfeito. O aluno entende a ideia de dependência sem virar “poluição” de versões e subdependências.
- **Menos chance de conflito:** em ambientes diferentes (laboratório, casa, notebook), amarrar versões pode quebrar instalação; deixar solto tende a instalar “o que funciona” mais facilmente.
- **Fácil de manter no início:** em projetos pequenos e em fase inicial, você quer flexibilidade.

Desvantagens:

- **Menos reprodutível:** dois alunos podem instalar versões diferentes do NumPy e, em casos raros, ter pequenas diferenças de comportamento, warnings, ou incompatibilidades com outras libs.
- **Pode quebrar no futuro:** se o NumPy lançar uma versão nova com mudanças relevantes, um código antigo pode parar de rodar igual.

Quando é “melhor”:

- Aulas, projetos iniciais, introdução ao ecossistema, turmas com computadores diferentes e necessidade de instalação simples.

10.4.2. Gerar requirements.txt automaticamente (`pip freeze > requirements.txt`)

Como fica:

- Aparece assim:
 - `numpy==2.4.2`

E muitas vezes aparecem outras linhas também, porque o freeze registra **tudo que está instalado** no ambiente, inclusive dependências indiretas.

O que isso significa tecnicamente:

- `numpy==2.4.2` quer dizer: “instale **exatamente** a versão 2.4.2 do NumPy”.
- Isso é chamado de **pinagem de versão** (version pinning).
- O objetivo é deixar o ambiente **reproduzível**, ou seja: “se você instalar isso, vai ficar igual ao meu”.

Vantagens (por que isso é bom e profissional):

- **Reprodutibilidade forte:** no laboratório, na casa, no servidor, no PC de outro dev — fica “igual”.
- **Controle e estabilidade:** você evita que uma atualização automática de biblioteca mude o comportamento do projeto.
- **Padrão de times e empresas:** para projetos que precisam rodar sempre da mesma forma (pipeline, servidor, produto), essa previsibilidade é valiosa.

Desvantagens (por que isso pode ser ruim, especialmente em aula):

- **Pode dar erro de instalação** em máquinas diferentes por vários motivos:
 - A versão exata pode não ter build disponível para aquele Python/Windows.
 - Pode exigir atualização do pip.
 - Pode conflitar com outras libs do ambiente.
 - Pode travar em versões que não são compatíveis com o hardware ou com a versão do Python do aluno.
- **Gera “ruído” desnecessário:** para ensino, o aluno aprende menos porque vê um “monte de versões” e perde o foco conceitual.

Quando é “melhor”:

- Projetos “de verdade” que vão para produção, projetos compartilhados em equipe, projetos que precisam ser reconstituídos exatamente (ex.: entregar para correção, rodar em máquina de um cliente, replicar em laboratório com muitos PCs).

10.4.3. Se o automático der erro, por que criar o manual?

Quando você usa o automático (pip freeze), você está pedindo para o outro computador repetir exatamente o seu ambiente. Se o outro PC for diferente (Python diferente, Windows diferente, permissões, pip antigo, etc.), **a chance de incompatibilidade aumenta**.

Então, criar o manual vira uma estratégia inteligente por dois motivos:

1. **Você reduz a rigidez**

Em vez de exigir `numpy==2.4.2`, você permite “qualquer NumPy adequado”. Isso aumenta as chances de instalação dar certo em ambientes variados.

2. **Você separa “essencial” do “acidental”**

O manual registra só o que o projeto realmente precisa.

O automático pode registrar coisas que você instalou “sem querer”, ou libs que não fazem parte do projeto, e isso vira entulho técnico.

10.4.4. **Por que no manual é numpy, mas no automático é numpy==2.4.2?**

- `numpy` (manual) → **dependência declarada de forma genérica**
“Preciso de NumPy, versão não fixada.”
- `numpy==2.4.2` (automático) → **dependência fixada por versão**
“Preciso exatamente desta versão, nem mais nova nem mais antiga.”

Do ponto de vista profissional:

- **Fixar versão** é mais “industrial” quando você precisa de controle e repetição.
- **Não fixar** é mais flexível e prático quando o objetivo é aprendizado, compatibilidade e facilidade de instalação.

10.4.5. **Conclusão prática (qual é melhor?)**

- Para aula (instalar uma biblioteca por vez, com foco didático): **o manual é melhor**, porque é mais simples, menos sujeito a falhas e mantém o aluno no conteúdo certo.
- Para um projeto que vai ser replicado exatamente em outra máquina/cliente/equipe: **o automático é mais profissional**, porque garante repetição fiel do ambiente — mas exige mais maturidade de configuração e compatibilidade.

11. Boas práticas (objetivas, sem entrar em redundância)

11.1. **Não versionar venv/**

A pasta venv/:

- é grande
- depende do sistema operacional
- pode ser recriada
- não é código-fonte do projeto

Logo, não deve ser enviada para repositório.

O correto é versionar:

- código (.py)
- requirements.txt

e ignorar o venv.

11.2. Isolamento por projeto

Um projeto → um ambiente virtual.

Isso evita:

- conflitos de versão
- contaminação de dependências
- “funciona aqui e não funciona ali”